

Material de apoyo al alumnado del ETEC

Módulo V: Programación estructurada

Puedes consultar el programa del módulo V en: https://www.ete.enp.unam.mx/programas_estudio

Nota: Este material contiene algunos conceptos generales que suelen abordarse en este módulo. Si quieres aprender más, se sugiere consultar:

- <https://learn.microsoft.com/es-es/cpp/c-language/c-language-reference>
- <https://www.w3schools.com/c/index.php> (en inglés)
- https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3071/mod_resource/content/1/UAPA-Fundamentos-Lenguaje-C/index.html

Hay material adicional disponible en el programa del módulo V.

Objetivo del módulo: Comprender la importancia y uso de la programación de computadoras como medio para solucionar problemas.

Estructura general de un programa

En C, los programas suelen tener la siguiente estructura:

- Declaración de bibliotecas a incluir
- Funciones del(a) programador(a)
- Función principal

Operadores

En lenguaje C tenemos distintos tipos de operadores:

Relacionales: sirven para comparar dos valores

Lógicos: sirven para verificar si un enunciado es verdadero o falso al aplicar conjunciones, disyunciones o negaciones

Aritméticos: permiten realizar las operaciones aritméticas suma, resta, multiplicación y módulo

De asignación: se utilizan al asignar valores a variables

También existe otro tipo de operadores, los operadores *bitwise* (lógicos a nivel de bits). No los abordaremos en este material, pero puedes investigar si te da curiosidad conocerlos.

Operadores relacionales

Operador	Función	Ejemplo
== (igual que)	Compara si dos valores son iguales. Regresa 1 si es verdadero.	<pre>a = 2; b = 2; if(a == b) printf("a y b son iguales");</pre>
!= (diferente de)	Compara si dos valores no son iguales. Regresa 1 si es verdadero.	<pre>a=4; if(a != 6) printf(a es diferente de 6);</pre>
>= (mayor o igual que)	Compara si un valor es mayor o igual que otro. Regresa 1 si es verdadero.	<pre>if(4 >= 4) printf("4 es mayor o igual que 4");</pre>
<= (menor o igual que)	Compara si un valor es menor o igual que otro. Regresa 1 si es verdadero.	<pre>if(2 <= 3) printf("2 es menor o igual que tres);</pre>
> (mayor que)	Compara si un valor es mayor que otro. Regresa 1 si es verdadero.	<pre>x = 6; if(x > 9) printf(x es mayor que 9);</pre>
< (menor que)	Compara si un valor es menor que otro. Regresa 1 si es verdadero.	<pre>x = 6; if(x < 9) printf(x es menor que 9);</pre>

Operadores lógicos

Operador	Función	Ejemplo
&& (AND, “y”)	Conjunción: comprueba si, de dos enunciados, ambos son verdaderos.	$2 > 1 \ \&\& \ 4 < 2$ // será verdadero $2 + 1 \leq 4 \ \&\& \ 3 + 2 < 3$ // será falso
 (OR, “o”)	Disyunción: comprueba si, de dos enunciados, al menos uno es verdadero.	$2 > 1 \ \ 4 < 2$ // será verdadero $2 + 1 \leq 4 \ \ 3 + 2 < 3$ // será verdadero
! (NOT, “no”)	Negación: si el valor al que se aplica era verdadero, lo invierte a falso y viceversa.	$!(2 > 1 \ \&\& \ 4 < 2)$ // será falso $!(2 + 1 \leq 4 \ \&\& \ 3 + 2 < 3)$ // será verdadero

Operadores aritméticos

Operador	Función	
+	Suma	$a = 9 + 12$ // a será 21
-	Resta	$a = 7 - 3$; // a será 4
*	Multiplicación	$a = 2 * 6$; // a será 12
/	División	$a = 64 / 16$; // a será 4
%	Módulo (residuo de una división)	$a = 2 \% 3$; // a será 1, pues el residuo de 2/3 es 1 (no se consideran los decimales)
++ (post-incremento)	Incrementa en 1 el valor de una variable después de usarse en un enunciado	<code>printf("a", a++);</code>
++ (pre-incremento)	Incrementa en 1 el valor de una variable antes de usarse en un enunciado	<code>printf("a", ++a);</code>
-- (post-decremento)	Disminuye en 1 el valor de una variable después de usarse en un enunciado.	<code>printf("a", a--);</code>
-- (pre-decremento)	Disminuye en 1 el valor de una variable antes de usarse en un enunciado.	<code>printf("a", --a);</code>

Operadores de asignación

Operador	Función	Ejemplo
=	Asigna un valor a una variable.	a = 2;
+=	Suma un valor a una variable y le asigna el resultado.	a += 3; // lo mismo que a = a + 3
-=	Resta un valor a una variable y le asigna el resultado.	a -= 7; // lo mismo que a = a - 7
/=	Divide una variable entre un valor y le asigna el resultado.	a /= 3; // lo mismo que a = a / 3
*=	Multiplica una variable por un valor y le asigna el resultado.	a *= 4; // lo mismo que a = a * 4
%=	Le realiza el módulo a una variable con un valor y le asigna el resultado.	a %= 2; // lo mismo que a = a % 2

Tipos de datos y máscaras

Tipos de datos primitivos

Los llamamos así porque no se componen de otro tipo de dato.

Tipo de dato	Función
int	Se utiliza para números enteros; que no tienen decimales.
float	Se utiliza para números reales.
double	Se utiliza para números reales. Puede almacenar mas dígitos que float.
char	Se utiliza para caracteres. Como <i>usualmente</i> un carácter suele ocupar 8 bits, este tipo de dato <i>generalmente</i> tendrá capacidad para almacenar 8 bits .

Es importante mencionar que la capacidad de bits que puede almacenar una variable con cada tipo de dato puede fluctuar entre cada computadora. Se utilizan los modificadores short, long y unsigned para modificar cómo utiliza el espacio cada tipo. Puedes consultar más acerca de eso en los enlaces sugeridos.

Máscaras

En funciones de entrada y salida que permiten dar formato (como scanf y printf), se utilizan las máscaras para elegir cómo queremos que se muestren nuestros datos.

Máscara	Representación
%i	Muestra un valor como número entero
%o	Muestra un valor como número en base 8 (octal)
%x	Muestra un valor como número en base 16 (hexadecimal)
%f	Muestra un valor como número real (float)
%lf	Muestra un valor como número real (double)
%u	Muestra un valor como número entero no negativo (unsigned int)
%c	Muestra un valor como caracter

Puedes experimentar al utilizar diferentes máscaras para una sola variable, como:

```
char a = 'e';
```

```
printf("Diferentes representaciones de la variable a: %i, %o, %x, %c", a, a, a, a);
```

Podemos especificar cuántos caracteres (el "ancho") queremos que utilice el valor al mostrarlo en la pantalla poniendo un número después del %. Por ejemplo:

```
Int a = 1234;
```

```
printf("%20i", a);
```

Imprime:

1234

(se desborda)

Asimismo, podemos especificar cuántos dígitos decimales deseamos mostrar en los números reales añadiendo un punto antes del %, seguido de la cantidad de dígitos. Por ejemplo:

```
double a = 4.12345789
```

```
printf("%.2lf\n",a); // podemos omitir especificar el ancho
```

```
printf("%5.3lf",a); // podemos no omitir especificar el ancho
```

Imprime:

```
4.12
```

```
4.123
```

Estructuras de control

Las estructuras de control sirven para darle “control” al flujo de nuestro programa. Tal vez recuerdes algunas del Módulo IV, aquí las repasaremos y veremos su sintaxis en C.

Estructura	Sintaxis	Descripción	Ejemplo
Condicional simple	<pre>If (condición) { acción si verdadero }</pre>	Realiza una acción solamente si una condición determinada se cumple. De lo contrario, la omite.	<pre>if (a > b) { printf("a > b"); }</pre>
Condicional doble	<pre>if (condición) { acción si verdadero } else { acción si falso }</pre>	Realiza una acción solamente si una condición determinada se cumple. De lo contrario, realiza una acción diferente.	<pre>if (a > b) { printf("a > b"); } else { printf("a < b"); }</pre>

Estructura	Sintaxis	Descripción	Ejemplo
Condicional múltiple	<pre> if (condición 1) { acción } else if (condición 2) { acción } else if (condición 3) ... </pre>	Realiza una acción solamente si una condición determinada se cumple. De lo contrario, realiza otra acción solamente si otra condición determinada se cumple, etc.	<pre> If (a == 1) { printf("a es 1"); } else if (a == 2) { printf("a es 2"); } else if (a == 3) { printf("a es 3"); } else if (a == 4) { printf("a es 4"); } else // si no se cumple ninguna condición { printf("a no es ni 1, ni 2, ni 3, ni 4); } </pre>
Iterativa indefinida	<pre> while (condición) { acción } - o - do { acción antes de verificar la condición por primera vez } while (condición); </pre>	Realiza repetidamente una acción siempre y cuando se cumpla una condición.	<pre> a=1; while (a < 6) { printf("a aún no es 6); a++; } // dejará de imprimir hasta que a sea 6 do { printf("Hola"); a++; // se garantiza que se realizará la acción al menos una vez } while (a >6); </pre>
Iterativa definida	<pre> for (variable de control; condición; incremento) { acción } </pre>	Realiza repetidamente una acción las veces especificadas.	<pre> int i; // entero for(i = 0; i <= 6; i++) { printf("%i", i); } // imprimirá 0123456 </pre>